

Parallel sorting and searching algorithms

General description

Sorting and searching are two of the most frequent algorithms in computer science. Below is a set of challenges designed to provide a "hands-on" experience in developing to sort or to search algorithms.

Sorting Algorithms

Shearsort is a parallelizable algorithm for sorting an $n \times n$ integer matrix in a snake-like order. With snake-like we mean that, after termination of shearsort, the rows of the given input matrix have been sorted in alternating directions. ShearSort takes an integer matrix M as input, which is assumed to be of size $n \times n$ (with n a positive integer). Then the following two steps are repeated $\lfloor \log_2(n) \rfloor + 1$ times:

1. Sort all rows of M in an alternating manner.
2. Sort all columns of M in ascending order.

A example:

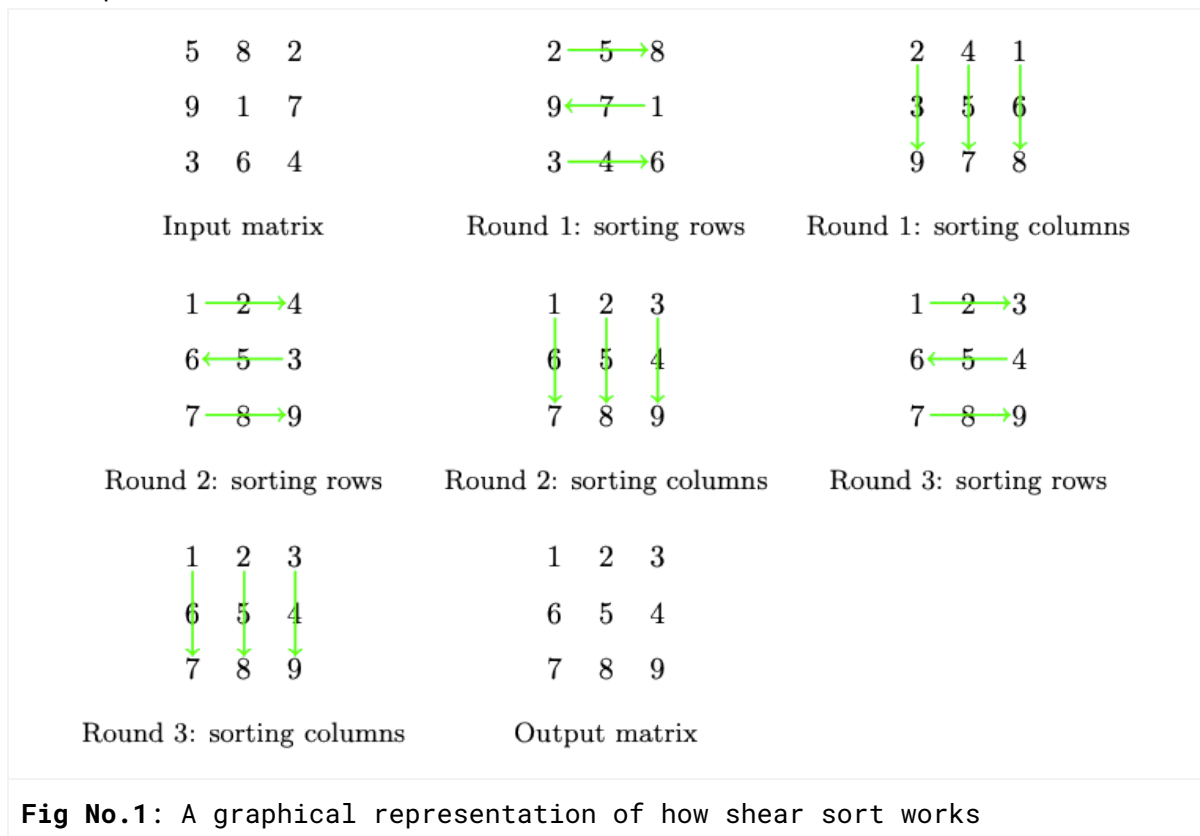


Fig No.1: A graphical representation of how shear sort works

The row and column sorts in every round can be performed in parallel. The implementations of sort-row and sort-column are left abstract, but could be chosen to be any sorting function.

```

1 // Sorts the row-th row of M in ascending order if ascending is true,
2 // or in descending order if ascending is false.
3 void sort-row(int[][] M, int row, bool ascending) {
4     ...
5
6 // Sorts the column-th column of M in ascending order.
7 void sort-column(int[][] M, int column) {
8     ...
9
10 // Sorts M in snake-like order, assuming that M is an  $n \times n$  matrix.
11 void shearsort(int n, int[][] M) {
12     repeat  $\lceil \log_2(n) \rceil + 1$  times {
13         for int tid = 0...n do in parallel {
14             sort-row(M, tid, tid % 2 == 0);
15         }
16         for int tid = 0...n do in parallel {
17             sort-column(M, tid);
18         }
19     }
20
21 // An alternative version of shearsort, that only uses sort-row.
22 void alternative-shearsort(int n, int[][] M) {
23     repeat  $\lceil \log_2(n) \rceil + 1$  times {
24         for int tid = 0...n do in parallel {
25             sort-row(M, tid, tid % 2 == 0);
26         }
27         transpose(M);
28         for int tid = 0...n do in parallel {
29             sort-row(M, tid, true);
30         }
31         transpose(M);
32     }

```

Fig No.2: Code structure and "functional cohesion"

Using C++ develop:

1. Shearsort in sequential version with arbitrary size and random values
2. Shearsort in parallel version.
3. Shearsort with sort-row, sort-column and transpose.
4. Compare implementations using asymptotic analysis and, show the speedup and the walltime metrics

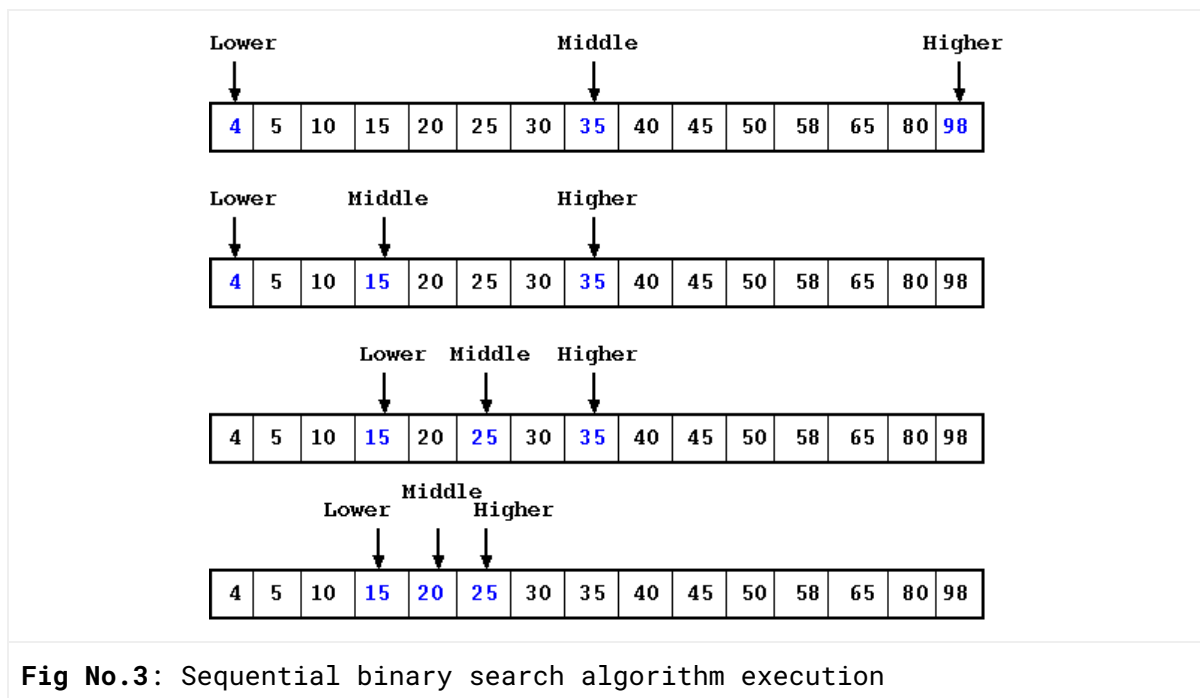
Search Algorithms

Parallel binary search algorithm (P-BSA) is based on dividing the sequence

each step, each of the P processors checks whether a value x is present in the subsequence assigned to that processor by comparing the value of x with the values at the start and end positions, marking the boundaries of the processor's subsequence.

If a processor finds the value x in its subsequence, it will notify the other processors about this, and then all P processors will search in that subsequence, which will be divided into other P subsequences, and the search will proceed to the next step. Unlike the serial version, where it checks if the searched element is in the middle position of the sequence, in the parallelized version, it checks if the element is in one of the start and end positions of the sequence (if it is in one of these positions, it means that we have found the element in the sequence).

A example of serial version:



Using C++ develop:

1. BSA with ordered random values (and non-ordered random values)
2. P-BSA with ordered random values (and non-ordered random values)
3. Compare implementations using asymptotic analysis and, show the speedup and the walltime metrics

Submit a PDF report detailing the implementations, along with a public repository on GitHub, GitLab, Bitbucket, or a similar platform. Additionally, you may include a video to complement the report.